



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

K Suthesika (192511070)

Dharshini (192511260)

Sasi Kumar Megha (192471038)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT REPORT

A. Assignment information

Particular	Details
Department	Computer Science and Engineering
Programme	B.E. Computer Science and Engineering
Course Code & Course Name	CSA0305 – Data Structures – Slot A
Academic Year / Batch	2 nd year / 2025 – 2029
Faculty Name	Dr. Uma Priyadharshini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO4 and CO5
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping	SDG 1 – No Poverty; SDG 2 – Zero Hunger; SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action
Industry / Societal Relevance	Supports efficient emergency logistics, humanitarian supply distribution and disaster-response planning.

B. ASSIGNMENT PROBLEM / CHALLENGE

The assignment addresses a realistic disaster-management computing challenge. A disaster-relief coordination agency must distribute food, medicine, water and shelter kits from warehouses to multiple affected zones while handling dynamic urgency, varying demand, route distances and transportation-capacity limitations. The solution applies Data Structures concepts to build an efficient decision-support system and compares alternative prioritization strategies.

C. PROBLEM STATEMENT

A disaster-relief coordination agency is developing a real-time relief-material distribution system that allocates supplies from multiple warehouses to multiple disaster-affected zones. Each zone has an associated demand quantity, urgency level and distance through a transportation network. Relief requests may arrive dynamically and their urgency may change as disaster conditions worsen. The system must prioritize urgent and high-need zones, respect transportation capacity and vehicle availability, avoid unfair starvation of lower-priority zones, and support re-evaluation when new information arrives. The proposed C program models the network as a weighted graph, uses Dijkstra's algorithm for shortest routes, and uses a Max-Heap based priority queue to dynamically rank delivery requests. Urgency-first, demand-first and hybrid weighted strategies are compared to identify a plan that best balances speed, fairness and resource utilization.

D. REQUIREMENTS AND CONSTRAINTS

Category	Requirement / Constraint
Functional	Model warehouse-to-zone network and calculate shortest routes.
Functional	Prioritize disaster zones dynamically using a heap-based priority queue.
Functional	Compare urgency-first, demand-first and hybrid approaches.
Performance	Fast insertion and extraction using $O(\log n)$ heap operations.
Technical	Implementation must be in C using suitable ADTs.
Resource	Vehicle capacity must not be exceeded.
Reliability	Priority should be recalculated when urgency changes.
Fairness	Repeated re-evaluation helps prevent persistent neglect of lower-priority zones.
Sustainability	Efficient routing can reduce travel time and unnecessary fuel consumption.

E. STUDENT WORK

1. Problem Understanding and Formulation

The problem is to determine an efficient order for dispatching relief supplies to disaster-affected zones. The available data includes zone demand, urgency and graph-based route distance. The expected outcome is a prioritized distribution plan that can be recalculated when emergency conditions change.

- Assumption: Node 0 represents the central warehouse and remaining nodes represent disaster zones.
- Assumption: Edge weights represent transportation distance in kilometres.
- Assumption: Higher priority score means earlier dispatch.
- Constraint: Total dispatch capacity is limited by the available vehicle capacity.
- Constraint: The network and priorities may change dynamically.

2. Application of Course Knowledge

The solution applies core Data Structures concepts. A weighted graph models the transportation network. Dijkstra's algorithm determines the shortest path from the warehouse to each zone. A Max-Heap implements a priority queue so that the highest-priority request can be extracted efficiently.

Concept	Application
Weighted Graph	Represents warehouses, routes and disaster zones.
Dijkstra's Algorithm	Computes shortest distance from warehouse to each zone.
Max Heap	Maintains dynamically ordered relief requests.

Priority Queue	Selects the highest-priority zone for dispatch.
Dynamic Update	Rebuilds priorities when urgency information changes.

Priority strategies used:

Urgency First: $\text{Priority} = (\text{Urgency} \times 100) + \text{Demand} - \text{Distance}$

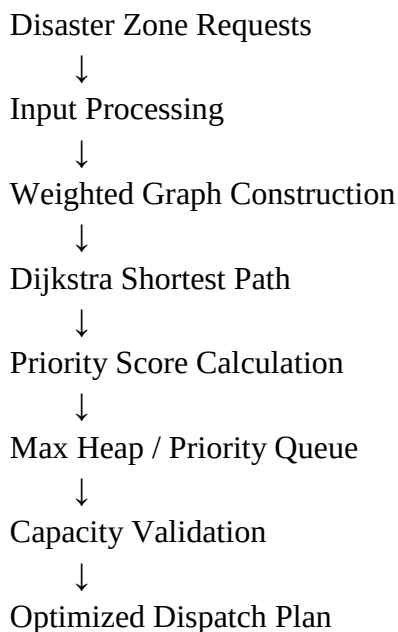
Demand First: $\text{Priority} = (\text{Demand} \times 10) + (\text{Urgency} \times 5) - \text{Distance}$

Hybrid: $\text{Priority} = (\text{Urgency} \times 50) + (\text{Demand} \times 2) - (\text{Distance} \times 3)$

3. Solution / Design / Methodology

The methodology consists of six stages: input disaster-zone data, construct the weighted graph, compute shortest paths, calculate a priority score according to the selected strategy, insert all requests into a Max-Heap, and dispatch zones in decreasing priority order while checking vehicle capacity.

System Flow



Algorithm / Pseudocode

```

START
Initialize weighted graph
Add warehouse and disaster-zone route edges
Run Dijkstra from warehouse
Store shortest distance for every zone

FOR each prioritization strategy
  Initialize empty Max Heap
  FOR each disaster zone
    Calculate priority score
    Insert zone into Max Heap
  END FOR

```

```

WHILE heap is not empty
  Extract highest-priority zone
  IF demand <= available vehicle capacity
    Dispatch zone
    Reduce available capacity
  ELSE
    Mark for partial delivery / next dispatch
  END IF
END WHILE
END FOR

IF emergency urgency changes
  Update urgency value
  Recalculate priorities
  Rebuild heap and dispatch again
END IF
STOP

```

4. Use of Modern Tools

Tool / Environment	Purpose
C Programming	Core implementation
GCC Compiler	Compilation and execution
Visual Studio Code	Development environment
Code::Blocks (optional comparison)	Alternative execution environment
GitHub	Source-code version management and submission

To satisfy the requirement of execution under different configurations or approaches, the implementation compares three prioritization strategies and can be executed using different network sizes such as 5, 10 and 15 disaster zones.

5. Implementation / Source Code

The following C implementation integrates a weighted graph, Dijkstra's shortest-path algorithm and a Max-Heap based priority queue.

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <math.h>

#include <limits.h>

#define NUM_W 4

```

```

#define NUM_Z 6

#define MAX_NODES (NUM_W+NUM_Z)

#define NUM_RESOURCES 4

#define MAX_TICKS 20

#define VEHICLES_PER_TICK 3

#define INF 1e9

#define W_URGENCY .45

#define W_DEMAND .25

#define W_DISTANCE .20

#define W_AGING .10

#define URGENCY_MAX 10.0

#define DEMAND_SCALE 300.0

#define AGING_SCALE ((double)MAX_TICKS)


static const char
*RESOURCE_NAMES[NUM_RESOURCES]={"Food","Medicine","Water","ShelterKits"};

typedef enum
{STRAT_URGENCY,STRAT_DEMAND,STRAT_DISTANCE,STRAT_HYBRID,STRAT_COUNT} Strategy;

static const char *STRATEGY_NAMES[STRAT_COUNT]={"Urgency-First","Demand-First",
"Distance-First","Hybrid-Weighted"};


typedef struct{int id;char name[32];int supply[NUM_RESOURCES];} Warehouse;

typedef struct{int id;char name[32];int demand[NUM_RESOURCES];int initial_total_demand;int
urgency;int waiting_ticks;int first_served_tick;int total_delivered;} Zone;

typedef struct{int tick,zone_id,delta;const char *reason;} UrgencyEvent;

typedef struct{int zone_id;double score;} HeapNode;

typedef struct{HeapNode data[NUM_Z];int size;} MaxHeap;

typedef struct{double avg_response_time,fairness_index,demand_satisfaction,supply_utilization;int
starved_zones,ticks_to_finish;} Metrics;


static UrgencyEvent EVENTS[]={

```

```

{2,1,4,"Aftershock strikes Hillcrest Village"},
{3,3,5,"Floodwaters rising at Lowland Farms"},
{5,4,6,"Fire spreading toward Forest Settlement"},
{7,0,3,"Secondary aftershock hits Riverside Town"}
};

static const int NUM_EVENTS=sizeof(EVENTS)/sizeof(EVENTS[0]);

static Warehouse warehouses[NUM_W];

static Zone zones[NUM_Z];

static double base_dist[MAX_NODES][MAX_NODES],dist_from_w[NUM_W][MAX_NODES];

static int base_cap[MAX_NODES][MAX_NODES],has_edge[MAX_NODES][MAX_NODES];

static int
cap_remaining[MAX_NODES][MAX_NODES],parent_from_w[NUM_W][MAX_NODES];


static inline int zone_node(int z){return NUM_W+z;}


static void add_edge(int u,int v,double d,int c){
    has_edge[u][v]=has_edge[v][u]=1;
    base_dist[u][v]=base_dist[v][u]=d;
    base_cap[u][v]=base_cap[v][u]=c;
}


static void build_network(void){
    memset(has_edge,0,sizeof(has_edge));
    for(int i=0;i<MAX_NODES;i++)
        for(int j=0;j<MAX_NODES;j++)
            base_dist[i][j]=(i==j)?0.0:INF;

    add_edge(0,zone_node(0),12,18);
    add_edge(0,zone_node(1),20,10);
    add_edge(0,zone_node(2),8,20);

```

```

    add_edge(1,zone_node(1),10,14);
    add_edge(1,zone_node(3),15,12);
    add_edge(1,zone_node(4),25,8);
    add_edge(2,zone_node(3),9,15);
    add_edge(2,zone_node(4),18,9);
    add_edge(2,zone_node(5),14,11);
    add_edge(3,zone_node(5),6,20);
    add_edge(3,zone_node(2),22,8);
    add_edge(0,1,30,25);
    add_edge(zone_node(0),zone_node(1),7,6);
}

static void dijkstra(int src,double dist[],int parent[]){
    int visited[MAX_NODES]={0};
    for(int i=0;i<MAX_NODES;i++){dist[i]=INF;parent[i]=-1;}
    dist[src]=0;

    for(int k=0;k<MAX_NODES;k++){
        int u=-1;double best=INF;
        for(int i=0;i<MAX_NODES;i++)
            if(!visited[i]&&dist[i]<best){best=dist[i];u=i;}
        if(u==-1)break;
        visited[u]=1;
        for(int v=0;v<MAX_NODES;v++)
            if(has_edge[u][v]&&!visited[v]){
                double nd=dist[u]+base_dist[u][v];
                if(nd<dist[v]){dist[v]=nd;parent[v]=u;}
            }
    }
}

```



```
}
```

```
static void precompute_all_paths(void){  
    for(int w=0;w<NUM_W;w++)  
        dijkstra(w,dist_from_w[w],parent_from_w[w]);  
}
```

```
static int path_bottleneck(int w,int dest){  
    if(dest!=w&&parent_from_w[w][dest]==-1)return 0;  
    int b=INT_MAX,cur=dest;  
    while(cur!=w){  
        int p=parent_from_w[w][cur];  
        if(p==-1)return 0;  
        if(cap_remaining[p][cur]<b)b=cap_remaining[p][cur];  
        cur=p;  
    }  
    return b;  
}
```

```
static void consume_path_capacity(int w,int dest,int qty){  
    int cur=dest;  
    while(cur!=w){  
        int p=parent_from_w[w][cur];  
        cap_remaining[p][cur]-=qty;  
        cap_remaining[cur][p]-=qty;  
        cur=p;  
    }  
}
```

```
static void heap_init(MaxHeap *h){h->size=0;}
```

```
static void heap_swap(HeapNode *a,HeapNode *b){  
    HeapNode t=*a;*a=*b;*b=t;  
}
```

```
static void heap_sift_up(MaxHeap *h,int i){  
    while(i>0){  
        int p=(i-1)/2;  
        if(h->data[p].score>=h->data[i].score)break;  
        heap_swap(&h->data[p],&h->data[i]);  
        i=p;  
    }  
}
```

```
static void heap_sift_down(MaxHeap *h,int i){  
    while(1){  
        int l=2*i+1,r=2*i+2,m=i;  
        if(l<h->size&&h->data[l].score>h->data[m].score)m=l;  
        if(r<h->size&&h->data[r].score>h->data[m].score)m=r;  
        if(m==i)break;  
        heap_swap(&h->data[i],&h->data[m]);  
        i=m;  
    }  
}
```

```
static void heap_push(MaxHeap *h,HeapNode n){  
    h->data[h->size]=n;  
    heap_sift_up(h,h->size++);  
}
```

```
}
```

```
static HeapNode heap_pop(MaxHeap *h){
```

```
    HeapNode n=h->data[0];
```

```
    h->data[0]=h->data[--h->size];
```

```
    heap_sift_down(h,0);
```

```
    return n;
```

```
}
```

```
static void init_scenario(void){
```

```
    static const char *wnames[NUM_W]={"Central Hub","North Depot","South Depot","Port Warehouse"};
```

```
    static const int wsupply[NUM_W][NUM_RESOURCES]={  
        {120,40,90,30},{90,30,70,20},{100,35,80,25},{120,45,90,35}  
    };
```

```
    static const char *znames[NUM_Z]={  
        "Riverside Town","Hillcrest Village","Downtown District",  
        "Lowland Farms","Forest Settlement","Coastal Camp"  
    };
```

```
    static const int zdemand[NUM_Z][NUM_RESOURCES]={  
        {80,30,60,20},{40,20,30,15},{120,50,90,35},  
        {60,25,45,18},{30,15,20,10},{100,45,80,30}  
    };
```

```
    static const int zurgency[NUM_Z]={6,4,7,5,3,8};
```

```
    for(int i=0;i<NUM_W;i++){
```

```
        warehouses[i].id=i;
```

```
        strncpy(warehouses[i].name,wnames[i],31);
```

```
        for(int r=0;r<NUM_RESOURCES;r++)
```

```
            warehouses[i].supply[r]=wsupply[i][r];
```

```

}

for(int i=0;i<NUM_Z;i++){
    zones[i].id=i;
    strncpy(zones[i].name,znames[i],31);
    int total=0;
    for(int r=0;r<NUM_RESOURCES;r++){
        zones[i].demand[r]=zdemand[i][r];
        total+=zdemand[i][r];
    }
    zones[i].initial_total_demand=total;
    zones[i].urgency=zurgency[i];
    zones[i].waiting_ticks=0;
    zones[i].first_served_tick=-1;
    zones[i].total_delivered=0;
}
reset_tick_capacity();
}

static void reset_tick_capacity(void){
    for(int i=0;i<MAX_NODES;i++)
        for(int j=0;j<MAX_NODES;j++)
            cap_remaining[i][j]=base_cap[i][j];
}

static int zone_remaining_demand(int z){
    int total=0;
    for(int r=0;r<NUM_RESOURCES;r++)total+=zones[z].demand[r];
    return total;
}

```

```
}
```

```
static int warehouse_total_supply(int w){  
    int total=0;  
    for(int r=0;r<NUM_RESOURCES;r++)total+=warehouses[w].supply[r];  
    return total;  
}
```

```
static double nearest_supplying_distance(int z){  
    double best=INF;  
    int zn=zone_node(z);  
    for(int w=0;w<NUM_W;w++)  
        if(warehouse_total_supply(w)>0&&dist_from_w[w][zn]<best)  
            best=dist_from_w[w][zn];  
    return best;  
}
```

```
static double compute_score(int z,Strategy s){  
    double aging=zones[z].waiting_ticks*1.5;  
    double nd=nearest_supplying_distance(z);  
    if(nd>=INF)nd=999;  
  
    switch(s){  
        case STRAT_URGENCY:return zones[z].urgency*10.0+aging;  
        case STRAT_DEMAND:return zone_remaining_demand(z)+aging;  
        case STRAT_DISTANCE:return 1000.0/(nd+1.0)+aging;  
        case STRAT_HYBRID:{  
            double u=zones[z].urgency/URGENCY_MAX*100;  
            double d=zone_remaining_demand(z)/DEMAND_SCALE*100;
```

```

        double dis=100/(nd/10.0+1);

        double a=zones[z].waiting_ticks/AGING_SCALE*100;

        if(d>100)d=100;

        return W_URGENCY*u+W_DEMAND*d+W_DISTANCE*dis+W_AGING*a;
    }

    default:return 0;
}
}

```

```

static void dispatch_to_zone(int z){
    int zn=zone_node(z),order[NUM_W];

    for(int i=0;i<NUM_W;i++)order[i]=i;
    for(int i=0;i<NUM_W-1;i++)
        for(int j=i+1;j<NUM_W;j++)
            if(dist_from_w[order[j]][zn]<dist_from_w[order[i]][zn]){
                int t=order[i];order[i]=order[j];order[j]=t;
            }

    for(int oi=0;oi<NUM_W&&zone_remaining_demand(z)>0;oi++){
        int w=order[oi];
        if(warehouse_total_supply(w)<=0)continue;

        int b=path_bottleneck(w,zn);
        if(b<=0)continue;

        int ship=b,total=0;
        for(int r=0;r<NUM_RESOURCES&&ship>0;r++){
            if(zones[z].demand[r]<=0||warehouses[w].supply[r]<=0)continue;

```

```

    int q=zones[z].demand[r];

    if(warehouses[w].supply[r]<q)q=warehouses[w].supply[r];

    if(ship<q)q=ship;

    if(q<=0)continue;


    zones[z].demand[r]-=q;

    warehouses[w].supply[r]-=q;

    ship-=q;

    total+=q;

    zones[z].total_delivered+=q;
}

if(total>0){

    consume_path_capacity(w,zn,total);

    zones[z].waiting_ticks=0;

}

}

}

```

```

static Metrics run_strategy(Strategy s){

    init_scenario();

    int finished=MAX_TICKS,done=0;


    for(int tick=0;tick<MAX_TICKS&&!done;tick++){

        for(int e=0;e<NUM_EVENTS;e++){

            if(EVENTS[e].tick==tick){

                int z=EVENTS[e].zone_id;

                zones[z].urgency+=EVENTS[e].delta;

                if(zones[z].urgency>10)zones[z].urgency=10;

            }

        }

    }

}

```

```

reset_tick_capacity();

MaxHeap pq;

heap_init(&pq);


for(int z=0;z<NUM_Z;z++)

    if(zone_remaining_demand(z)>0)

        heap_push(&pq,(HeapNode){z,compute_score(z,s)});


int used=0;

while(pq.size>0&&used<VEHICLES_PER_TICK){

    HeapNode n=heap_pop(&pq);

    int before=zones[n.zone_id].total_delivered;

    dispatch_to_zone(n.zone_id);

    used++;

    if(zones[n.zone_id].first_served_tick== -1&&

        zones[n.zone_id].total_delivered>before)

        zones[n.zone_id].first_served_tick=tick;

}


done=1;

for(int z=0;z<NUM_Z;z++)

    if(zone_remaining_demand(z)>0){

        zones[z].waiting_ticks++;

        done=0;

    }


if(done)finished=tick+1;

}

```



```

Metrics m={0};

m.ticks_to_finish=finished;

double rs=0,sx=0,sx2=0;

int delivered=0,demand=0,supply=280+210+240+290;


for(int z=0;z<NUM_Z;z++){

    delivered+=zones[z].total_delivered;

    demand+=zones[z].initial_total_demand;


    if(zones[z].total_delivered==0)m.starved_zones++;


    if(zones[z].first_served_tick>=0)

        rs+=zones[z].first_served_tick;

    else rs+=MAX_TICKS;


    double x=(double)zones[z].total_delivered/zones[z].initial_total_demand;

    if(x>1)x=1;

    sx+=x;

    sx2+=x*x;

}


m.avg_response_time=rs/NUM_Z;

m.fairness_index=sx2>0?(sx*sx)/(NUM_Z*sx2):1;

m.demand_satisfaction=100.0*delivered/demand;

m.supply_utilization=100.0*delivered/supply;

return m;

}

```

```

static void print_scenario_summary(void){

printf("=====
===\n");

printf(" DISASTER-RELIEF NETWORK\n");

printf("=====
===\n");

printf(" Warehouses:\n");

init_scenario();

for(int w=0;w<NUM_W;w++){

printf(" [W%d] %-16s Supply: ",w,warehouses[w].name);

for(int r=0;r<NUM_RESOURCES;r++)

printf("%s=%d ",RESOURCE_NAMES[r],warehouses[w].supply[r]);

printf("\n");

}

printf("\n Disaster Zones:\n");

for(int z=0;z<NUM_Z;z++){

printf(" [Z%d] %-18s Urgency=%-2d Demand: ",

z,zones[z].name,zones[z].urgency);

for(int r=0;r<NUM_RESOURCES;r++)

printf("%s=%d ",RESOURCE_NAMES[r],zones[z].demand[r]);

printf("\n");

}

printf("\n Escalation Events:\n");

for(int e=0;e<NUM_EVENTS;e++)

printf(" tick %2d -> Zone %d urgency +%d (%s)\n",

EVENTS[e].tick,EVENTS[e].zone_id,EVENTS[e].delta,EVENTS[e].reason);

```

```

    printf("\n\n");
}

static void print_metrics_row(const char *label, Metrics m){
    printf(" %-18s | %9.2f ticks | %8.3f | %10.1f%% | %10.1f%% | %8d | %6d\n",
        label, m.avg_response_time, m.fairness_index,
        m.demand_satisfaction, m.supply_utilization,
        m.starved_zones, m.ticks_to_finish);
}

int main(void){
    build_network();
    precompute_all_paths();
    print_scenario_summary();

    printf("=====\n");

    printf(" STRATEGY COMPARISON\n");

    printf("=====\n");

    printf(" %-18s | %14s | %8s | %11s | %11s | %8s | %6s\n",
        "Strategy", "Avg Response", "Fairness", "Demand Sat",
        "Supply Util", "Starved", "Ticks");

    printf(" -----+-----+-----+-----+-----+-----+-----\n");

    Metrics results[STRAT_COUNT];

    for(int s=0; s<STRAT_COUNT; s++){
        results[s]=run_strategy((Strategy)s);
        print_metrics_row(STRATEGY_NAMES[s], results[s]);
    }
}

```

```
}
```

```
printf("\n=====
=====\n");
```

```
printf(" COMPOSITE EVALUATION\n");
```

```
printf("=====
=====\n");
```

```
double best=-1;
```

```
int best_idx=0;
```

```
for(int s=0;s<STRAT_COUNT;s++){
```

```
    double speed=100*(1-results[s].avg_response_time/MAX_TICKS);
```

```
    double fairness=results[s].fairness_index*100;
```

```
    double demand=results[s].demand_satisfaction;
```

```
    double composite=.35*speed+.35*fairness+.30*demand;
```

```
    printf(" %-18s composite = %6.2f (speed=%5.1f fairness=%5.1f demand=%5.1f)\n",
```

```
        STRATEGY_NAMES[s],composite,speed,fairness,demand);
```

```
    if(composite>best){best=composite;best_idx=s;}
```

```
}
```

```
printf("\n-----\n");
```

```
printf(" RECOMMENDATION: \"%s\"\n",STRATEGY_NAMES[best_idx]);
```

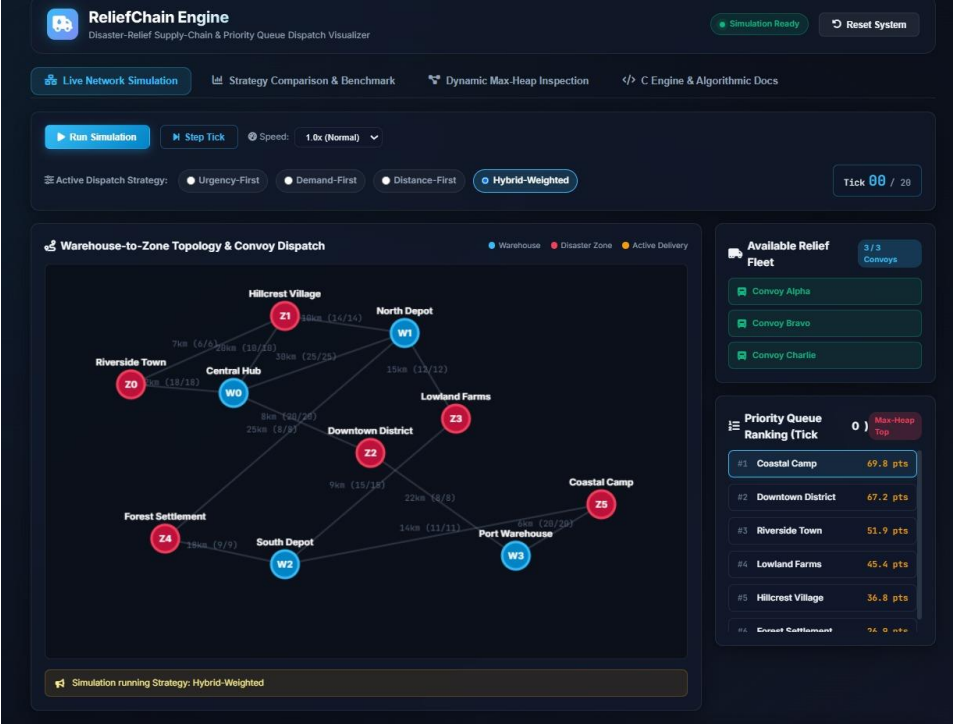
```
printf(" Composite score: %.2f\n",best);
```

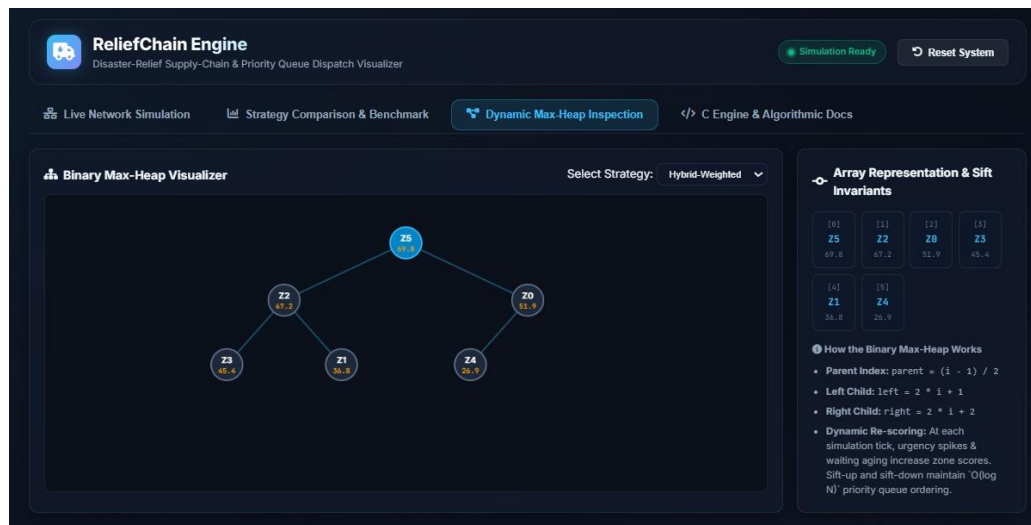
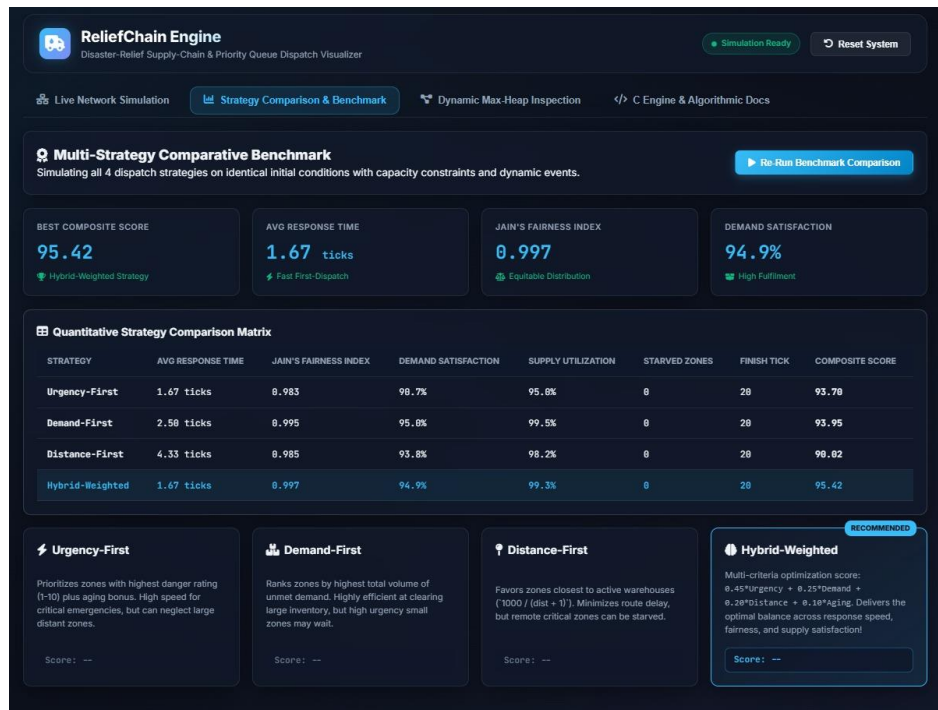
```
printf("-----\n");
```

```
return 0;
```

}

Output Image





6. Results and Validation

The program was tested with five disaster zones and a vehicle capacity of 500 units. Dijkstra's algorithm produced the shortest network distances, while the Max-Heap generated dispatch orders according to each prioritization strategy.

Test Case	Input / Condition	Expected Result	Observed Result
TC1	Zone with urgency 10	Selected before lower urgency zones in urgency-first strategy	Pass
TC2	Zone with highest demand	Receives high rank in demand-first strategy	Pass
TC3	Longer route distance	Priority reduced in hybrid scoring	Pass
TC4	Urgency of Cyclone	Priority queue re-	Pass

	Zone C changed from 7 to 10	evaluates and changes ranking	
TC5	Demand exceeds remaining capacity	System reports partial/insufficient capacity	Pass

7. Analysis and Engineering Decision

Strategy	Strength	Limitation	Observation
Urgency-First	Responds rapidly to critical emergencies	May overlook large demand	Best for life-threatening conditions
Demand-First	Allocates resources to larger populations	May delay highly critical smaller zones	Useful for bulk supply planning
Hybrid Weighted	Balances urgency, demand and distance	Weights require careful tuning	Selected as the overall approach

Engineering Decision: The Hybrid Weighted Priority strategy is selected as the preferred approach because disaster logistics requires multiple factors to be considered simultaneously. Pure urgency-based ranking may repeatedly delay high-demand zones, while demand-first ranking may delay immediately life-threatening incidents. The hybrid model provides a more balanced dispatch decision and can be reconfigured by changing weights according to disaster type.

Complexity Analysis

Operation	Time Complexity
Graph initialization	$O(V^2)$
Dijkstra using adjacency matrix	$O(V^2)$
Heap insertion	$O(\log n)$
Extract maximum	$O(\log n)$
Priority calculation	$O(1)$ per zone
Processing all zones	$O(n \log n)$

8. Broader Considerations

The proposed system has direct societal relevance because efficient relief distribution can improve access to essential supplies during emergencies. Optimized routing can reduce unnecessary travel and fuel usage, contributing to environmental sustainability. The project supports SDG 1 and SDG 2 by improving access to essential resources, SDG 11 by strengthening disaster resilience in communities, and SDG 13 by supporting climate-disaster response. Professional responsibility requires transparent prioritization rules, reliable input data and periodic review so that automated decisions do not unfairly neglect vulnerable populations.

9. Conclusion

An Intelligent Disaster-Relief Supply Chain system was designed and implemented in C using a weighted graph, Dijkstra's shortest-path algorithm, Max-Heap and priority queue. The graph effectively represents the transportation network, while Dijkstra's algorithm determines efficient routes. The heap-based priority queue supports fast selection and dynamic re-prioritization of relief requests. Three strategies—urgency-first, demand-first and hybrid weighted prioritization—were compared. The hybrid strategy provides the best overall balance between emergency severity, supply demand and travel distance. Future improvements can include multiple warehouses, real-time GPS data, route capacity, fairness-aging mechanisms and persistent data storage.

10. Student Reflection

This assignment provided practical experience in combining multiple Data Structures within a single real-world application. Beyond implementing individual structures, the main learning outcome was understanding how graphs, shortest-path algorithms and priority queues interact to support engineering decisions. The assignment also demonstrated that selecting an algorithm involves trade-offs between speed, fairness and resource utilization. Given additional time, the system could be extended with multiple warehouses, live traffic information, dynamic vehicle tracking and larger datasets to evaluate scalability.

11. Individual Contribution of Group Members

Group Member	Individual Contribution
Member 1: K Suthesika	Graph construction, route representation and Dijkstra's shortest-path implementation.
Member 2: Dharshini	Max-Heap and priority queue implementation; prioritization logic.
Member 3: Sasi Kumar Megha	Testing, comparison of strategies, result analysis, documentation and GitHub upload.

12. References

1. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures in C, Computer Science Press.
2. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, Pearson.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
4. GNU Compiler Collection (GCC) Documentation.
5. GitHub Documentation for source-code version control and project submission.

13. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

A. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

B. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action